
DESAFIO DMC · DESENVOLVIMENTO DE SOFTWARE

Relatório de Apresentação

GrupoJap Rentals — Aplicação web de gestão de aluguer de veículos

Relatório técnico entregue com o projeto do Desafio DMC, no âmbito do processo de recrutamento do GrupoJap.

Empresa	GrupoJap
Desafio	DMC — Desenvolvimento de Software
Projeto	GrupoJap Rentals (painel de administração)
Candidato	Nuno
Tecnologias	ASP.NET Core MVC (.NET 10), C#, Entity Framework Core 10, SQL Server, xUnit
Repositório	github.com/nunoomigueIII/GrupoJap_Interview_Project

Índice

1.	Sumário executivo	3
2.	Contexto e objetivos do desafio	3
3.	Stack tecnológica e justificação das escolhas	3
4.	Arquitetura da solução e organização do código	4
5.	Modelo de dados e base de dados	6
6.	Funcionalidades implementadas por módulo	7
7.	Validações e regras de negócio	9
8.	Cálculo automático do estado do veículo	10
9.	Frontend e experiência de utilização	11
10.	Testes automatizados	12
11.	Controlo de versões (Git)	13
12.	Cumprimento dos critérios de avaliação	14
13.	Limitações conhecidas e melhorias futuras	14
14.	Instruções de instalação e execução	15
15.	Conclusão	16

1. Sumário executivo

O GrupoJap Rentals é um painel de administração em ASP.NET Core MVC (.NET 10) que cumpre os quatro requisitos funcionais do desafio: registo de veículos, de clientes, de contratos de aluguer e listagens com o estado Disponível/Alugado calculado automaticamente. A persistência usa SQL Server com Entity Framework Core 10 em Code-First, com 6 migrações versionadas.

Todas as validações pedidas no enunciado estão implementadas no servidor, reforçadas na base de dados (índices únicos) e espelhadas no cliente. Para além do mínimo, a solução acrescenta:

- **Bloqueio de contratos sobrepostos** para o mesmo veículo, com datas inclusivas — a regra que evita que um carro seja alugado duas vezes.
- **Autenticação e autorização** com ASP.NET Core Identity, restrita ao perfil Admin, com bloqueio de conta após 5 tentativas falhadas.
- **Painel inicial (dashboard)** com indicadores da frota e contratos recentes.
- **Interface em três idiomas** (PT, EN, ES), com 305 textos geridos a partir da própria aplicação.
- **Modo noturno, design responsivo** e mensagens de confirmação em todas as operações.
- **87 métodos de teste xUnit** (≈133 casos com os `InlineData`), que cobrem regras de negócio, validações e controladores.

A solução está organizada em três projetos — aplicação web, biblioteca de núcleo e testes — com as regras de negócio isoladas em classes puras e testáveis.

2. Contexto e objetivos do desafio

O Desafio DMC pede uma aplicação web para o aluguer de veículos, um cenário real do setor automóvel em que o GrupoJap atua. O prazo de entrega é de 5 dias, seguido de uma apresentação ao painel com explicação das escolhas técnicas e da arquitetura.

O enunciado define quatro funcionalidades principais:

1. Registo de veículos (marca, modelo, matrícula, ano de fabrico, tipo de combustível).
2. Registo de clientes (nome completo, email, telefone, carta de condução).
3. Registo de contratos de aluguer, que associam um cliente a um veículo com datas de início e fim.
4. Listagens de veículos, clientes e contratos, com o estado de cada veículo (disponível ou alugado).

Os requisitos técnicos são ASP.NET MVC ou ASP.NET Core MVC, C#, HTML/CSS/JavaScript, SQL Server e um mecanismo de acesso a dados (ADO.NET, Entity Framework ou Dapper). Os testes automatizados com xUnit ou NUnit são valorizados como diferencial.

O objetivo pessoal neste projeto foi entregar mais do que um CRUD: uma aplicação coerente, segura e testada, próxima do que seria colocado em produção num contexto empresarial.

3. Stack tecnológica e justificação das escolhas

A stack segue à letra os requisitos técnicos do enunciado, escolhendo em cada ponto a opção mais atual e mais adequada ao critério de avaliação 4 (Entity Framework Code-First).

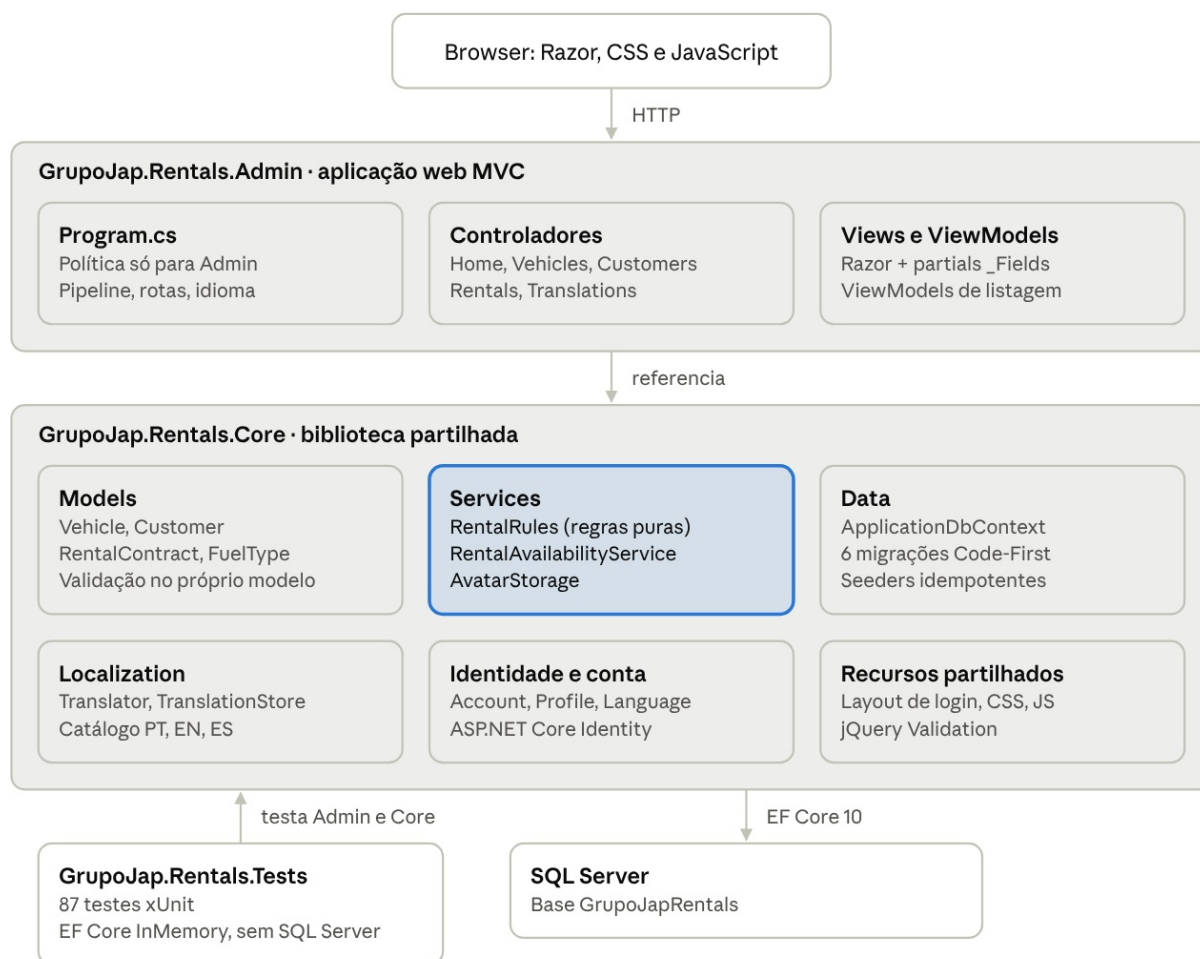
Camada	Tecnologia	Porquê
Plataforma	ASP.NET Core MVC, .NET 10	Versão atual e multiplataforma; o enunciado aceita qualquer versão.
Linguagem	C# com Nullable ativo	Os avisos de nulos detetam erros em tempo de compilação.
Acesso a dados	Entity Framework Core 10	ORM pedido explicitamente nos critérios de avaliação; consultas LINQ tipadas.
Abordagem	Code-First com migrações	O modelo C# é a fonte de verdade e o esquema evolui de forma versionada.
Base de dados	SQL Server (Express em desenvolvimento)	Requisito do enunciado.
Autenticação	ASP.NET Core Identity	Solução oficial, com hashing de palavras-passe, bloqueio e perfis.
Frontend	Razor, HTML, CSS próprio, JavaScript sem framework	Interface leve, sem dependências de build.
Validação no cliente	jQuery Validation + Unobtrusive	Reutiliza as mesmas DataAnnotations do servidor.
Testes	xUnit + EF Core InMemory	Framework aceite pelo enunciado; testes rápidos e sem SQL Server.
Ferramentas	dotnet -ef 10.0.12 como ferramenta local	Versão fixada em .config/dotnet-tools.json, igual para toda a equipa.

Foram deliberadamente evitadas dependências externas desnecessárias. Toda a interface (layout, tabelas, formulários, modo noturno) usa CSS e JavaScript próprios, o que mantém o projeto simples de executar e de rever.

4. Arquitetura da solução e organização do código

A solução (GrupoJap.slnx) divide-se em três projetos: a aplicação web, uma biblioteca de núcleo com o domínio e a infraestrutura, e os testes. Esta separação mantém as regras de negócio fora dos controladores e permite testá-las sem servidor web nem SQL Server.

As regras de negócio vivem no Core, partilhadas e testáveis



arquitetura da solução · 3 projetos

O Admin contém apenas o que é específico da interface de gestão; tudo o que é reutilizável (modelos, acesso a dados, serviços, traduções, login) está no Core, uma Razor Class Library que também partilha vistas e recursos estáticos.

Padrão MVC aplicado

- **Model**: entidades de domínio com `DataAnnotations` e `IValidatableObject`; ViewModels próprios para as listagens, para não expor as entidades diretamente às vistas.
- **View**: Razor com Tag Helpers (`asp-for`, `asp-validation-for`), layout comum e partials reutilizáveis (`_Fields`, `_Notices`, `_Eyebrow`).
- **Controller**: controladores finos, `sealed`, com dependências injetadas por construtor primário, rotas por atributos e ações assíncronas.

Decisões de arquitetura

- **Regras puras separadas do acesso a dados**: `RentalRules` é uma classe estática sem dependências (sobreposição, contrato ativo, estado). O `RentalAvailabilityService` aplica a mesma regra numa consulta EF Core.
- **Injeção de dependências**: toda a configuração está num método de extensão `AddRentalsCore`, que regista o `DbContext`, o `Identity`, os serviços e as traduções. O `Program.cs` fica curto e legível.

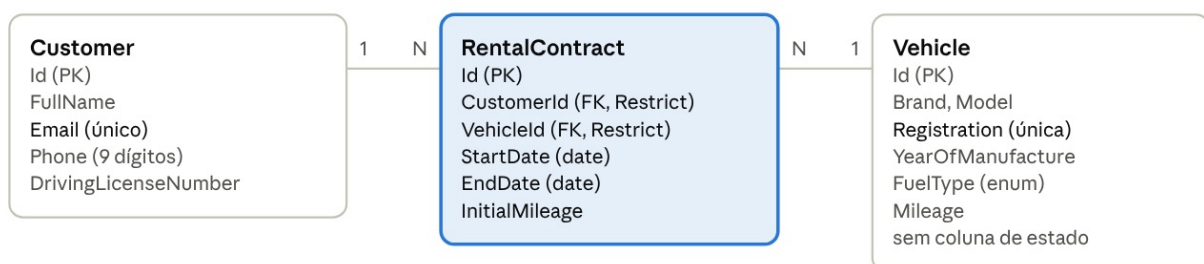
- **Segurança por omissão:** uma política de fallback exige o perfil Admin em todas as rotas; só o login e o acesso negado são anônimos. Uma rota nova fica protegida mesmo que alguém se esqueça do atributo `[Authorize]`.
- **Sem camada de repositório genérico:** o `DbContext` do EF Core já implementa os padrões Unit of Work e Repository. Acrescentar outra camada aumentaria o código sem benefício num projeto deste tamanho.

5. Modelo de dados e base de dados

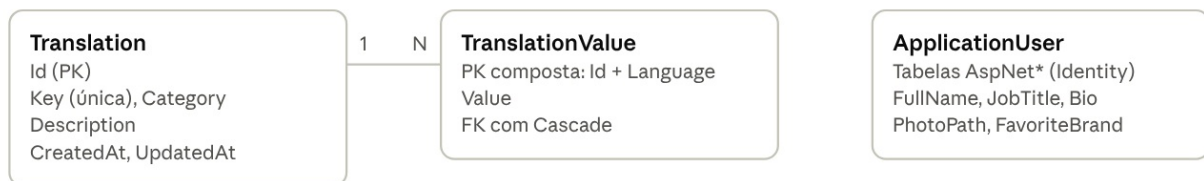
O modelo foi definido em C# e gerado no SQL Server com Entity Framework Core em Code-First: as classes são a fonte de verdade e cada alteração ao esquema é uma migração versionada.

Cada contrato liga um cliente a um veículo; o estado não é guardado

Domínio do desafio



Suporte: traduções e identidade



modelo de dados · 6 entidades

Um cliente pode ter vários contratos e um veículo pode ter vários contratos ao longo do tempo; cada contrato pertence a exatamente um cliente e um veículo.

Decisões de modelação

- **DateOnly para as datas do contrato:** mapeado para o tipo `date` do SQL Server. Um aluguer conta-se em dias, por isso guardar horas e fusos horários só criaria erros nas comparações.
- **Campos de contrato anuláveis em C# (`int?`, `DateOnly?`) mas `[Required]`:** permite distinguir "campo não preenchido" de "valor zero" e mostrar a mensagem certa ao utilizador, em vez de um `0` ou `01/01/0001` silencioso.
- **Combustível como `enum`** (Gasolina, Gasóleo, Híbrido, Elétrico, GPL, Outro), guardado como inteiro. O valor `Unspecified = 0` nunca é aceite, o que obriga a uma escolha explícita.
- **Índices únicos** em `Vehicles.Registration`, `Customers.Email` e `Translations.Key`, declarados com o atributo `[Index]` ou na Fluent API.
- **DeleteBehavior.Restrict** nas duas chaves estrangeiras do contrato: apagar um cliente ou veículo com histórico de contratos é impedido pela própria base de dados, preservando o histórico.

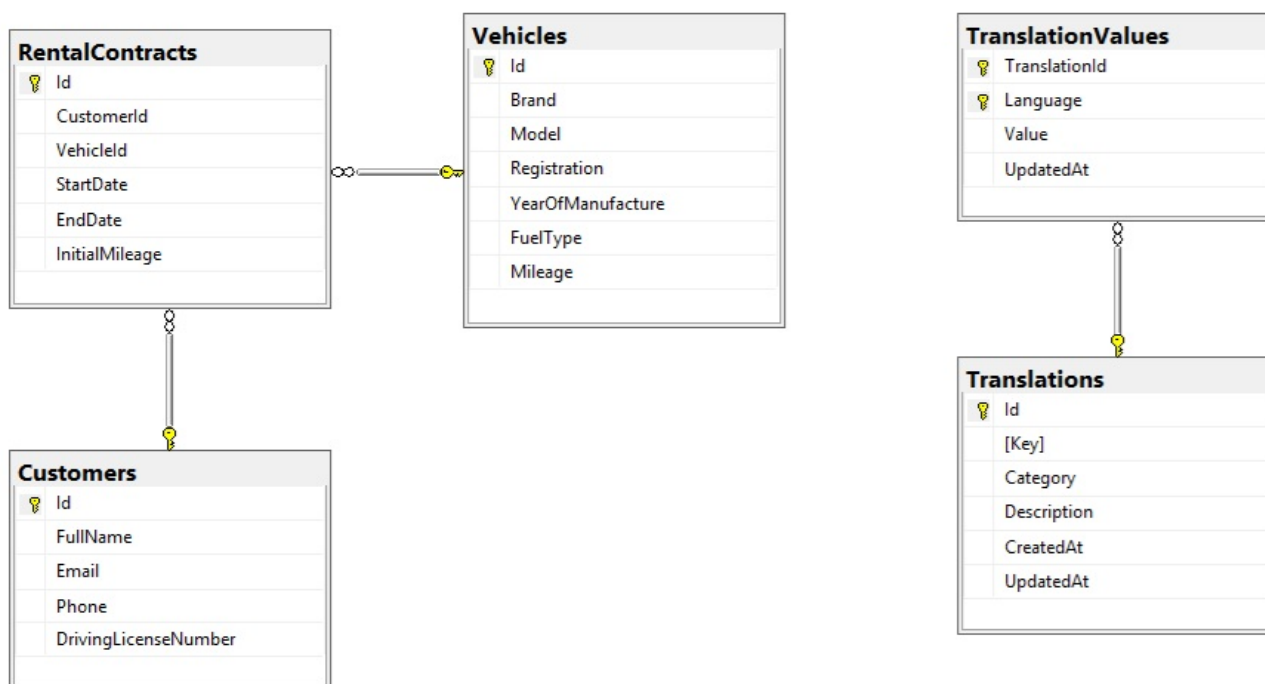
- **Limites de tamanho** (`StringLength`) em todos os textos, que geram colunas `nvarchar(n)` em vez de `nvarchar(max)`.
- **Quilometragem do veículo**: campo adicional (`Mileage`, 0 a 2 000 000 km), registrado no veículo e usado nos dados de demonstração como quilometragem inicial dos contratos.

Migrações

As 6 migrações (`InitialCreate`, `LimitPhoneLength`, `AddIdentity`, `AddUserProfileFields`, `AddVehicleMileage`, `AddTranslations`) mostram a evolução incremental do esquema. A base de dados pode ser recriada em qualquer ambiente com um único comando `dotnet ef database update`.

Esquema real no SQL Server

O diagrama seguinte foi exportado do SQL Server Management Studio a partir da base `GrupoJapRentals`, depois de aplicadas as migrações. Confirma que o esquema gerado pelo EF Core corresponde ao modelo: `RentalContracts` liga-se a `Customers` e `Vehicles` por chaves estrangeiras, `TranslationValues` depende de `Translations`, e as tabelas `AspNet*` são as do ASP.NET Core Identity. A tabela `__EFMigrationsHistory` regista as migrações aplicadas.



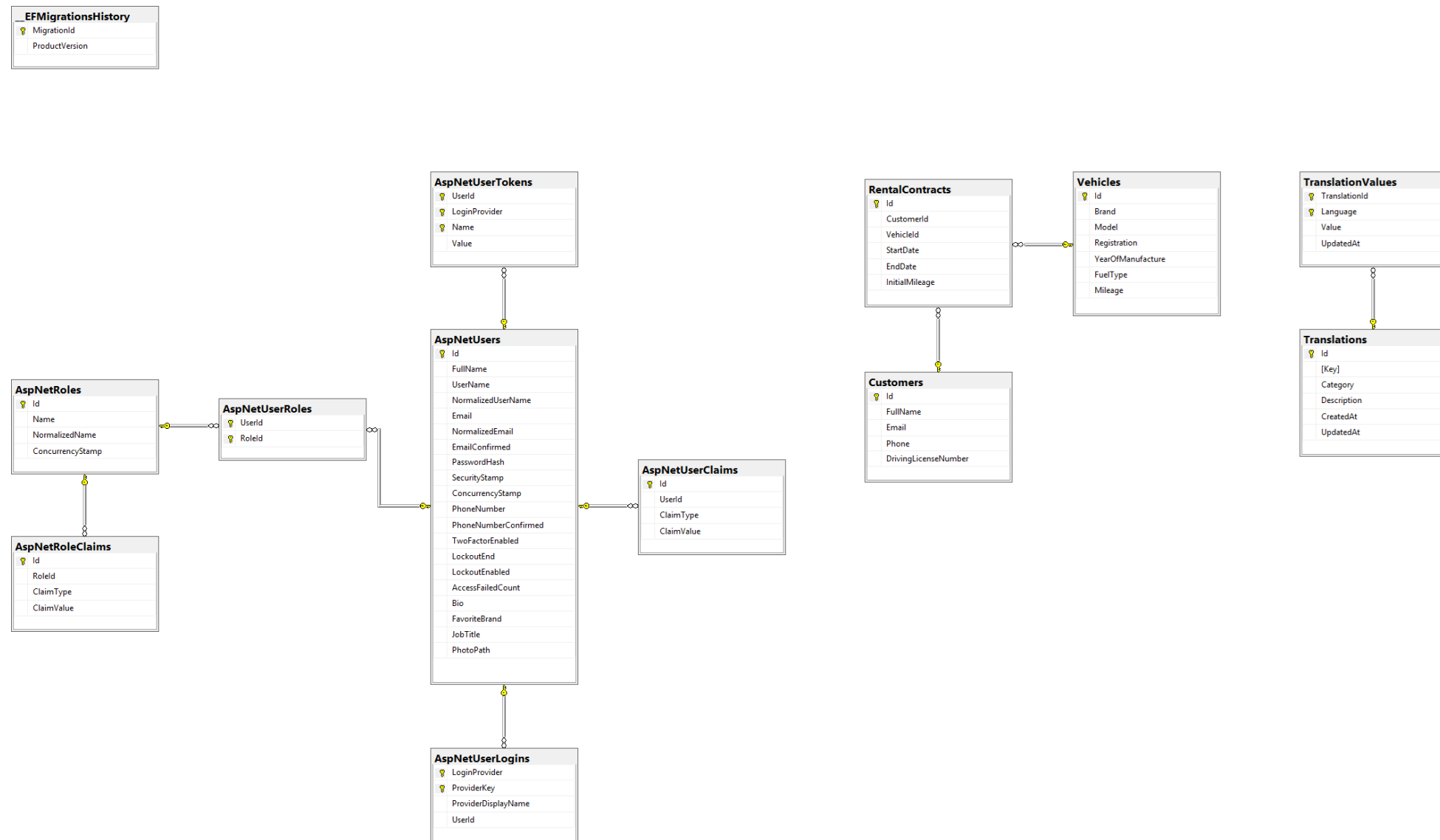
Tabelas do domínio e das traduções (excerto do diagrama exportado do SSMS). O diagrama completo, com as tabelas do Identity, está na página seguinte.

6. Funcionalidades implementadas por módulo

Cada entidade do enunciado tem um CRUD completo — listar, criar, editar e eliminar com página de confirmação — com rotas REST legíveis definidas por atributos.

Módulo	Rotas principais	O que faz
Painel inicial	/	Total de veículos, veículos disponíveis, contratos ativos, clientes e os 5 contratos mais recentes.
Veículos	/vehicles, /vehicles/create,	Marca, modelo, matrícula, ano, combustível e

Diagrama E-R completo da base GrupoJapRentals (SQL Server Management Studio)



Inclui as tabelas do domínio, das traduções, do ASP.NET Core Identity (AspNet*) e o histórico de migrações (__EFMigrationsHistory).

	<code>/vehicles/{id}/edit</code> , <code>/vehicles/{id}/delete</code>	quilometragem; estado Disponível/Alugado na lista.
Clientes	<code>/customers</code> , <code>/customers/create</code> , ...	Nome, email, telefone e carta de condução; nº de contratos e indicação de aluguer em curso.
Contratos	<code>/rentals</code> , <code>/rentals/create</code> , ...	Associa cliente e veículo com datas e quilometragem inicial; estado Agendado, Em curso ou Concluído; pesquisa instantânea na lista.
Traduções	<code>/translations</code>	Pesquisa, filtro por categoria ou textos em falta, paginação e edição dos textos em PT/EN/ES.
Conta e perfil	<code>/account/login</code> , <code>/profile</code> , <code>/profile/password</code>	Login, logout, perfil com foto, alteração de palavra-passe.
Idioma	<code>/language/{code}</code>	Troca de idioma guardada num cookie durante 1 ano.

Alguns detalhes que melhoram o uso diário:

- **Atalhos entre módulos:** o formulário de contrato aceita `?vehicleId=` e `?customerId=` para chegar já com o veículo ou o cliente pré-selecionado.
- **Valores por omissão sensatos:** um contrato novo começa hoje e termina amanhã; um veículo novo assume o ano atual.
- **Normalização dos dados:** matrículas e cartas de condução em maiúsculas, emails em minúsculas, espaços removidos do telefone. Assim, `aa-00-bb` e `AA-00-BB` são tratados como a mesma matrícula.
- **Integridade referencial:** não é possível eliminar um veículo ou cliente com contratos associados. A página de confirmação mostra quantos contratos existem e a base de dados reforça a regra com `DeleteBehavior.Restrict`.
- **Mensagens de feedback:** cada operação termina com uma mensagem de sucesso ou erro (padrão Post-Redirect-Get com `TempData`).
- **Dados de demonstração:** em desenvolvimento, a aplicação cria 14 veículos, 3 clientes e 5 contratos (em curso, agendado e concluído) se a frota estiver vazia, para uma demonstração imediata.

7. Validações e regras de negócio

Todas as validações do ponto 4 do enunciado estão cumpridas, e cada uma é aplicada no servidor — a validação JavaScript é apenas um complemento de usabilidade, nunca a única proteção.

A validação está organizada em três camadas:

1. **Modelo:** `DataAnnotations` (`[Required]`, `[Range]`, `[RegularExpression]`, `[EmailAddress]`) e `IValidatableObject` para regras que dependem da data atual ou de vários campos.
2. **Controlador:** regras que precisam de consultar a base de dados — unicidade, existência do cliente e do veículo, sobreposição de contratos.
3. **Base de dados:** índices únicos em `Registration` e `Email` e chaves estrangeiras com `Restrict`. Se dois pedidos simultâneos passarem a validação, o SQL Server rejeita o segundo; a aplicação apanha os erros 2601/2627 e mostra a mesma mensagem amigável.

Mapeamento requisito → implementação

Requisito do enunciado	Implementação	Onde
Veículo: marca, modelo	<code>[Required]</code> + limites de tamanho	<code>Vehicle</code>

Veículo: marca, modelo, matrícula obrigatórios	[Required] + limites de tamanho	Vehicle
Veículo: ano e combustível obrigatórios	[Range(1, ...)] no ano; o valor Unspecified do enum é rejeitado	Vehicle.Validate
Ano de fabrico não posterior ao atual	Comparação com DateTime.Today.Year	Vehicle.Validate
Matrícula única	Verificação prévia + índice único + captura do erro SQL	VehiclesController, [Index(IsUnique)]
Cliente: todos os campos obrigatórios	[Required] em nome, email, telefone e carta	Customer
Email único	Normalizado em minúsculas, verificação prévia + índice único	CustomersController
Email com formato válido (extra)	[EmailAddress]	Customer
Telefone só com números e formato válido	Expressão ^9[0-9]{8}\$ (telemóvel português: 9 dígitos a começar por 9)	PhoneNumbers.PortuguesePattern
Contrato: cliente e veículo obrigatórios	[Required] + verificação de que existem na BD	RentalContract, RentalsController
Datas e quilometragem inicial obrigatórias	[Required]; quilometragem >= 0	RentalContract
Início não anterior à data atual	Comparação com o dia de hoje (DateOnly)	RentalContract.Validate
Fim posterior ao início	end <= start é rejeitado	RentalContract.Validate
Estado do veículo automático	Calculado a partir dos contratos (ver secção 8)	RentalRules, listagens

Regras adicionais, para além do enunciado

- **Sem sobreposição de contratos:** um veículo não pode ter dois contratos com períodos que se cruzem (detalhe na secção 8).
- **Edição de contratos já iniciados:** um contrato em curso pode ser corrigido (por exemplo, prolongar o fim) desde que a data de início original se mantenha. Sem esta regra, a validação "início não anterior a hoje" impediria qualquer edição de contratos em curso.
- **Proteção contra overposting:** cada ação POST usa [Bind] com a lista explícita de campos aceites.
- **Proteção CSRF:** todos os formulários POST usam [ValidateAntiForgeryToken].
- **Mensagens traduzidas:** as mensagens de erro são chaves de tradução, por isso aparecem no idioma escolhido.

8. Cálculo automático do estado do veículo

O estado Disponível/Alugado não é guardado na base de dados: é calculado em cada consulta a partir dos contratos. Esta é a decisão técnica central do projeto, porque elimina por construção o risco de o estado ficar desatualizado.

Porquê não guardar o estado

Uma coluna `IsRented` na tabela de veículos exigiria um processo (job agendado, trigger ou atualização manual) para a mudar quando um contrato começa ou termina. Se esse processo falhasse ou se atrasasse, o veículo apareceria com o estado errado. Ao derivar o estado das datas dos contratos, a informação é sempre correta no momento em que é mostrada, sem qualquer processo em segundo plano.

As regras, num só sítio

As regras vivem na classe estática `RentalRules`, sem acesso a dados, o que as torna fáceis de ler e de testar. Todas as datas são inclusivas e usam `DateOnly`, que representa um dia sem hora nem fuso horário.

```
// Um contrato está ativo no dia indicado quando início <= dia <= fim
public static bool IsActiveOn(DateOnly start, DateOnly end, DateOnly day)
=> start <= day && end >= day;

// Dois períodos inclusivos sobrepõem-se quando inícioA <= fimB e fimA >= inícioB
public static bool PeriodsOverlap(DateOnly startA, DateOnly endA, DateOnly startB,
DateOnly endB)
=> startA <= endB && endA >= startB;
```

- **Veículo:** Alugado se existir pelo menos um contrato com `início <= hoje <= fim`; caso contrário, Disponível.
- **Contrato:** Agendado se ainda não começou, Em curso se hoje está dentro do período, Concluído se já terminou (enum `RentalStatus`).
- **Cliente:** a lista indica se o cliente tem um aluguer em curso.

Eficiência

Nas listagens, o estado é calculado dentro da própria consulta SQL: o LINQ

`vehicle.RentalContracts.Any(r => r.StartDate <= today && r.EndDate >= today)` é traduzido pelo EF Core numa subconsulta `EXISTS`. Assim, a lista completa obtém-se numa única ida à base de dados, sem o problema N+1 (uma consulta extra por veículo). As consultas de leitura usam `AsNoTracking()` e projetam diretamente para ViewModels, carregando só as colunas necessárias.

Prevenção de sobreposições

A mesma fórmula de sobreposição é usada pelo serviço `RentalAvailabilityService` ao criar ou editar um contrato. A verificação é feita com um `AnyAsync` no SQL Server (uma única ida à BD, apoiada pelo índice da chave estrangeira `VehicleId`). Na edição, o próprio contrato é excluído da comparação, para que não entre em conflito consigo mesmo.

Exemplo com datas inclusivas: se um veículo está alugado de 10 a 15, um novo contrato de 15 a 20 é rejeitado (o dia 15 é partilhado), mas um de 16 a 20 é aceite.

9. Frontend e experiência de utilização

A interface foi desenhada como um painel de gestão de frota, com identidade visual inspirada no GrupoJap, sem frameworks de CSS: cerca de 460 linhas de CSS e 86 linhas de JavaScript próprios.

- **Layout:** menu lateral com navegação ativa, barra superior com seletor de idioma, modo noturno, data atual e avatar do utilizador.
- **Responsivo:** em ecrãs pequenos o menu lateral passa a gaveta, que fecha com o botão, com Escape ou

ao clicar fora; as tabelas têm scroll horizontal próprio.

- **Modo noturno:** segue o tema do sistema por omissão e guarda a escolha do utilizador no browser. O tema é aplicado antes de a página ser desenhada, para evitar um "flash" de cor.
- **Estados visuais:** etiquetas coloridas para Disponível, Alugado, Agendado, Em curso e Concluído; matrículas com aspeto de chapa; estados vazios com ligação direta para criar o primeiro registo.
- **Formulários:** campos partilhados entre Criar e Editar através de `partials (_Fields.cshtml)`, validação imediata no cliente e mensagens junto a cada campo.
- **Pesquisa instantânea** na lista de contratos, por cliente, veículo ou matrícula, sem recarregar a página.
- **Acessibilidade:** `aria-label`, `aria-expanded` e `aria-pressed` nos controlos, ícones decorativos com `aria-hidden` e navegação por teclado.

Internacionalização

O painel está disponível em português (idioma base), inglês e espanhol. Em vez de ficheiros `.resx`, os 305 textos vivem na base de dados (tabelas `Translations` e `TranslationValues`) e podem ser editados no próprio painel, sem novo deploy.

- Um catálogo em código (`TranslationCatalog`) define os valores iniciais; ao arrancar, só as chaves em falta são inseridas, sem apagar edições feitas pelos utilizadores.
- Uma cache em memória (`TranslationStore`) evita consultar a base de dados em cada texto. É descartada ao guardar uma alteração e expira sozinha ao fim de 10 segundos. O recarregamento usa um lock com dupla verificação, seguro com pedidos simultâneos.
- Ordem de recurso de cada texto: BD no idioma atual → catálogo no idioma atual → BD em português → catálogo em português → a própria chave. A interface nunca fica com textos vazios.
- O `Translator` substitui o `IStringLocalizerFactory` da framework, por isso as mensagens das `DataAnnotations` também são traduzidas.

10. Testes automatizados

O projeto inclui 87 métodos de teste xUnit — 70 [Fact] e 17 [Theory] com 63 conjuntos de dados — num total de cerca de 133 casos executados. Os testes não precisam de SQL Server: cada um usa uma base EF Core em memória nova, por isso são independentes entre si e correm em segundos.

Ficheiro de testes	Nº de testes	O que garante
<code>RentalsControllerTests</code>	16	Conflito de períodos nas fronteiras, períodos adjacentes aceites, cliente/veículo inexistente, início no passado, fim inválido, km negativos, edição sem conflito consigo própria, contratos já iniciados
<code>ModelValidationTests</code>	13	Campos obrigatórios, ano não futuro, combustível, telefone, email, datas e quilometragem
<code>TranslatorTests</code>	10	Ordem de recurso das traduções e cache
<code>VehiclesControllerTests</code>	9	Matrícula duplicada ao criar e editar, normalização, eliminação bloqueada, estado na lista
<code>CustomersControllerTests</code>	9	Email duplicado sem distinguir maiúsculas, email e telefone inválidos, eliminação bloqueada
<code>TranslationCatalogTests</code>	6	Catálogo completo nos 3 idiomas, marcadores <code>{0}</code> coerentes, todas

		as chaves usadas no código existem
TranslationSeederTests	5	O seeder não apaga edições e remove chaves obsoletas
AvatarStorageTests	5	Deteção do tipo real do ficheiro e limite de tamanho
LanguageControllerTests	4	Cookie de idioma e proteção contra open redirect
RentalRulesTests	3	Sobreposição e contrato ativo com datas inclusivas, estado Agendado/Em curso/Concluído
HomeControllerTests	3	Indicadores do painel e contratos recentes
ValidationLocalizationTests	2	Mensagens de validação traduzidas
RentalAvailabilityServiceTests	2	Serviço de verificação de sobreposição

Os testes de controladores usam um contexto próprio (`ControllerTestContext`) que monta o controlador como num pedido real: base de dados em memória, traduções reais e `TempData`. Também reproduz a validação de modelo que o MVC faz antes de chamar a ação, para testar o fluxo completo e não apenas métodos isolados.

Os testes com `[Theory]` são usados sobretudo nas fronteiras das datas, onde costumam estar os erros: períodos que partilham um único dia, períodos adjacentes e datas de fim iguais à de início.

Comando para executar: `dotnet test`.

11. Controlo de versões (Git)

O projeto é versionado em Git no ramo `nuno-dev`, com commits pequenos e descritivos ao longo do desenvolvimento, como definido no plano de trabalho do README.

Repositório: github.com/nunoomiguell/GrupoJap_Interview_Project

Boas práticas aplicadas:

- **.gitignore** exclui `bin/`, `obj/`, `.vs/`, ficheiros `*.user` e a pasta `uploads/` com fotos de utilizadores.
- **Migrações versionadas:** o histórico do esquema acompanha o código, permitindo recriar a base de dados em qualquer ambiente.
- **Ferramentas fixadas:** `dotnet-ef` como ferramenta local, para que todos usem a mesma versão.
- **Credenciais fora do código partilhado:** o README recomenda User Secrets ou variáveis de ambiente fora do ambiente de desenvolvimento.

O plano de 5 dias foi seguido por fases, o que se reflete na ordem das migrações:

Data	Migração	Fase
3 out 2026	AddTranslations	Experiência de utilização (idiomas)
2 out 2026	AddVehicleMileage	Contratos (quilometragem)
2 out 2026	AddUserProfileFields	Perfil do utilizador
2 out 2026	AddIdentity	Autenticação
1 out 2026	LimitPhoneLength	Clientes (telefone)
1 out 2026	InitialCreate	Domínio e persistência

12. Cumprimento dos critérios de avaliação

A tabela liga cada critério da secção 6 do enunciado às evidências concretas no projeto.

Critério	Evidência no projeto
1. C# e .NET	.NET 10, C# moderno: construtores primários, <code>sealed</code> , <code>record</code> , <code>DateOnly</code> , pattern matching, <code>Nullable</code> ativo, <code>async/await</code> em todo o acesso a dados.
2. Estruturas de dados e algoritmos	Sobreposição de intervalos em $O(1)$ por par; estado calculado com <code>EXISTS</code> numa só consulta; cache de traduções em dicionários (procura $O(1)$) com lock de dupla verificação.
3. Desenvolvimento web com ASP.NET	Rotas por atributos legíveis (<code>/vehicles/{id}/edit</code>), restrições de tipo (<code>{id:int}</code>), PRG com <code>TempData</code> , <code>ViewModels</code> , <code>partials</code> reutilizáveis, <code>Tag Helpers</code> .
4. SQL Server e Entity Framework	Code-First com 6 migrações, índices únicos, chaves estrangeiras com <code>Restrict</code> , chave composta, projeções e <code>AsNoTracking()</code> .
5. Git	Histórico com commits pequenos, <code>.gitignore</code> , migrações e ferramentas versionadas (secção 11).
6. POO	Entidades encapsulam a sua validação (<code>IValidatableObject</code>); regras puras em <code>RentalRules</code> ; serviço injetado <code>RentalAvailabilityService</code> ; classes <code>sealed</code> por omissão; extensões de configuração.
7. Análise de problemas	Plano por fases no README; decisões documentadas (formato de telefone, datas inclusivas, estado derivado).
8. Raciocínio algorítmico	Fórmula de sobreposição única e testada nas fronteiras; edição de contratos em curso tratada como caso especial.
9. Pensamento crítico e criatividade	Prevenção de sobreposições, autenticação, multi-idioma editável, modo noturno, dashboard, dados de demonstração.
10. Testes automatizados (diferencial)	87 métodos <code>xUnit</code> , ≈ 133 casos, incluindo testes de controladores com BD em memória.

13. Limitações conhecidas e melhorias futuras

Dentro do prazo de 5 dias, foram priorizados os requisitos do enunciado e a qualidade das regras de negócio. Os pontos abaixo são limitações conscientes, cada uma com a solução que seria adotada a seguir.

Limitação atual	Impacto	Melhoria proposta
A verificação de sobreposição é feita na aplicação, não na base de dados	Dois pedidos exatamente simultâneos para o mesmo veículo poderiam, em teoria, passar ambos	Transação com isolamento <code>Serializable</code> ou lock por veículo (<code>sp_getapplock</code>) ao criar/editar contratos
Os testes usam EF Core InMemory	Não validam índices únicos nem a tradução exata para SQL	Testes de integração com SQL Server real (Testcontainers ou LocalDB)
Não há fecho de contrato (devolução)	A quilometragem final não é registada e a quilometragem do veículo não é	Ação "Devolver veículo" com km final, validação $\text{km final} \geq \text{km inicial}$ e atualização

	atualizada	do veículo
A quilometragem inicial não é comparada com a do veículo	É possível registar um valor inferior ao atual	Sugerir por omissão a km do veículo e validar que não é inferior
O telefone aceita apenas telemóveis portugueses (9 dígitos a começar por 9)	Exclui fixos e números internacionais	Biblioteca <code>libphonenumber</code> com indicativo de país
A carta de condução não tem validação de formato nem de validade	Aceita qualquer texto até 30 caracteres	Validar formato e data de validade da carta à data do contrato
Sem paginação nem filtros nas listas de veículos e clientes	Lento com milhares de registos	Paginação no servidor e filtros por estado, marca e combustível (já existe nas traduções)
"Hoje" usa o relógio do servidor	Dependente do fuso horário do servidor	Injetar <code>TimeProvider</code> (.NET 8+), o que também facilita os testes

Outras evoluções naturais para o negócio:

- **Preços e faturação:** tarifa diária por veículo e cálculo do valor do contrato.
- **Calendário de disponibilidade:** vista mensal por veículo com os contratos agendados.
- **Auditoria:** registo de quem criou ou alterou cada contrato (`CreatedBy`, `UpdatedAt`).
- **Arquitetura em camadas mais explícita:** se o domínio crescer, separar os serviços de aplicação dos controladores (por exemplo, `VehicleService`) para reduzir a lógica nos controladores.
- **CI/CD:** pipeline no GitHub Actions para compilar e correr os testes em cada push.

14. Instruções de instalação e execução

Pré-requisitos: .NET 10 SDK e SQL Server (Express ou outra instância). A connection string de desenvolvimento usa `localhost\SQLEXPRESS` com autenticação do Windows e pode ser alterada em `src/GrupoJap.Rentals.Admin/appsettings.Development.json`.

1. Restaurar pacotes e ferramentas: `dotnet restore` e `dotnet tool restore`.
2. Criar a base de dados: `dotnet ef database update --project src/GrupoJap.Rentals.Core --startup-project src/GrupoJap.Rentals.Admin`.
3. Arrancar a aplicação: `dotnet run --project src/GrupoJap.Rentals.Admin`.
4. Abrir `http://localhost:5169` e entrar com o administrador de desenvolvimento (`admin@admin.com`).
5. Correr os testes: `dotnet test`.

No primeiro arranque são criados automaticamente os perfis, o administrador, os textos de tradução e os dados de demonstração. Todas estas operações são idempotentes: não duplicam nem alteram dados existentes.

Roteiro sugerido para a demonstração

1. Login e painel inicial com os indicadores da frota.
2. Lista de veículos: estados Disponível e Alugado já calculados a partir dos contratos de demonstração.
3. Criar um veículo com matrícula repetida ou ano futuro, para mostrar as validações.
4. Criar um cliente com email repetido ou telefone inválido.

5. Criar um contrato sobreposto a outro do mesmo veículo e ver a rejeição; depois um contrato válido.
6. Voltar à lista de veículos e ver o estado atualizado.
7. Mudar o idioma para inglês, ativar o modo noturno e editar uma tradução.
8. Correr `dotnet test` e mostrar os testes a passar.

15. Conclusão

O GrupoJap Rentals cumpre todos os requisitos funcionais e técnicos do Desafio DMC e o diferencial de testes automatizados. A decisão de derivar o estado dos veículos a partir dos contratos, em vez de o guardar, garante que a informação mostrada está sempre correta.

Para além do pedido, a aplicação previne alugueres sobrepostos, protege o acesso com autenticação e perfis e oferece uma interface multi-idioma editável e responsiva. As regras de negócio estão isoladas e cobertas por cerca de 133 casos de teste.

As limitações identificadas na secção 13 são conhecidas e têm um caminho claro de evolução. O projeto foi construído como um ponto de partida sólido para um sistema real de gestão de alugueres no GrupoJap.